Task 4: AWS EC2 Start/Stop Automation Using Lambda and CloudWatch/EventBridge

Objective

GOAL: Automate the start/stop of EC2 instances using Lambda and CloudWatch services.

Create the required IAM policies and roles.

Create a function under AWS Lambda that determines the start/stop process of a specific EC2 instance.

Test the function created under Lambda.

Create rules inside CloudWatch service that trigger the function created in Lambda using cron.

Requirements

- AWS EC2 instance running
- AWS IAM
- AWS Lambda
- Amazon CloudWatch/EventBridge
- Python 3.14 runtime
- Boto3
- Specific EC2 instance

Procedure

Step 1 – Identify the EC2 Instance

An AWS EC2 instance was used as the target instance for the automation.

The instance ID used for the Lambda function was:

```
i-0eb1c9832726e5355
```

The Lambda function was configured to perform start and stop operations only on this specific EC2 instance.

Step 2 – Create the IAM Policy

An IAM policy was created to provide the Lambda function with the required permissions to communicate with the EC2 service.

The following EC2 permissions were added:

```
ec2:DescribeInstances
ec2:StartInstances
ec2:StopInstances
```

The policy JSON used was:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:DescribeInstances",
                "ec2:StartInstances",
                "ec2:StopInstances"
            ],
            "Resource": "*"
        }
    ]
}
```

`DescribeInstances` allows Lambda to check the current state of the EC2 instance, while `StartInstances` and `StopInstances` allow Lambda to start and stop the instance.



Step 3 – Create the IAM Role

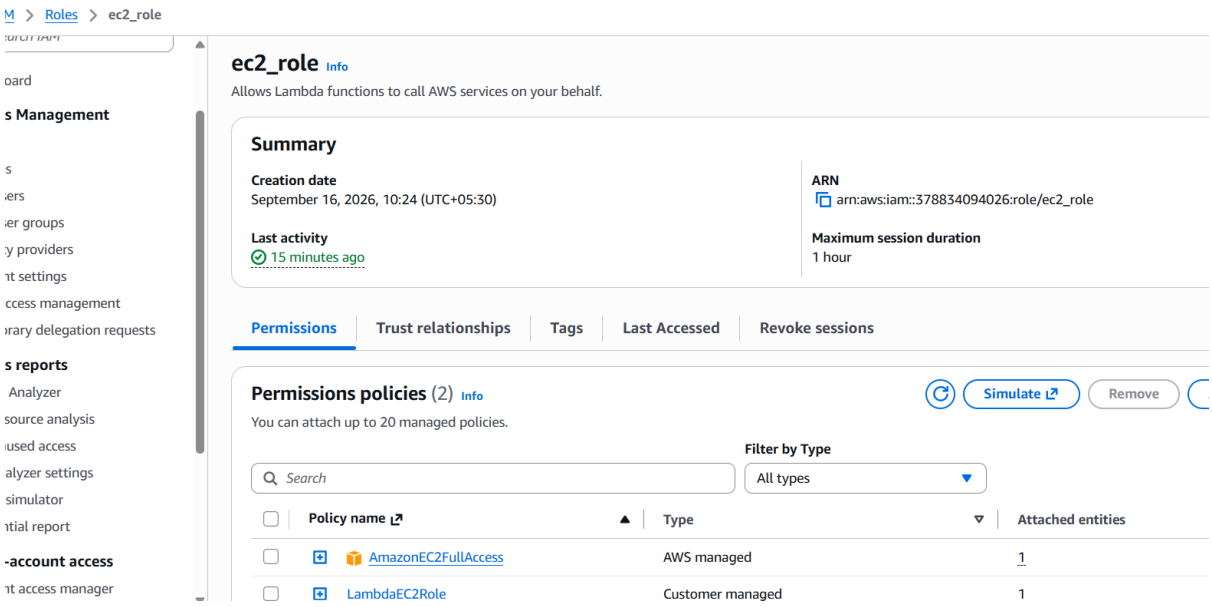
An IAM role was created for the Lambda function.

The role name used was:

```
ec2_role
```

The EC2 policy created in the previous step was attached to the `ec2_role`.

This role allows the Lambda function to perform the required EC2 operations.



Step 4 – Create the AWS Lambda Function

An AWS Lambda function was created using:

```
Function name: EC2StatusChecker
Runtime: Python 3.14
Architecture: x86_64
Execution role: ec2_role
```

The Lambda function was created successfully.

Step 5 – Configure the Lambda Function

The Lambda function was configured using Python and the Boto3 library.

The following code was added:

```
import boto3

ec2 = boto3.client('ec2')

INSTANCE_ID = 'i-0eb1c9832726e5355'


def lambda_handler(event, context):

    response = ec2.describe_instances(
        InstanceIds=[INSTANCE_ID]
    )
```

```
state = response['Reservations'][0]['Instances'][0]['State']['Name']

print(f"Current EC2 state: {state}")

if state == 'stopped':

    ec2.start_instances(
        InstanceIds=[INSTANCE_ID]
    )

    print(f"EC2 instance {INSTANCE_ID} is stopped. Starting it.")

    return {
        'statusCode': 200,
        'message': 'EC2 instance started',
        'instance_id': INSTANCE_ID
    }

elif state == 'running':

    ec2.stop_instances(
        InstanceIds=[INSTANCE_ID]
    )

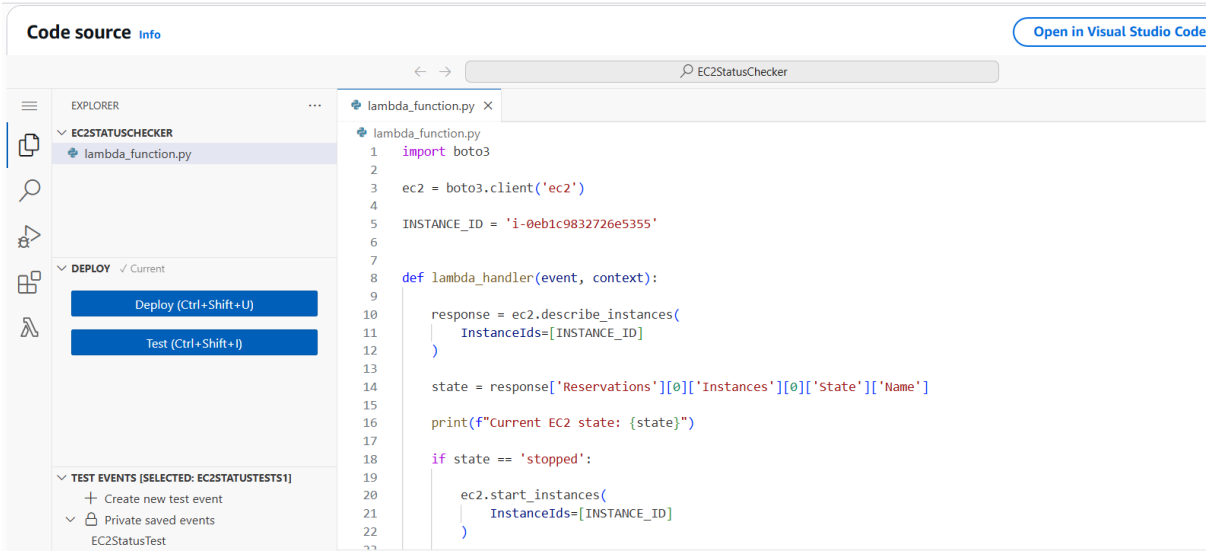
    print(f"EC2 instance {INSTANCE_ID} is running. Stopping it.")

    return {
        'statusCode': 200,
        'message': 'EC2 instance stopped',
        'instance_id': INSTANCE_ID
    }

else:

    return {
        'statusCode': 200,
        'message': f'EC2 instance is currently {state}',
        'instance_id': INSTANCE_ID
    }
```

The code was deployed successfully.



Step 6 – Test the Lambda Function

A Lambda test event was created to test the function.

The test event JSON was:

```
{}
```

No action value was required because the Lambda function checks the actual EC2 state and automatically decides whether to start or stop the instance.

The Lambda function was tested successfully.

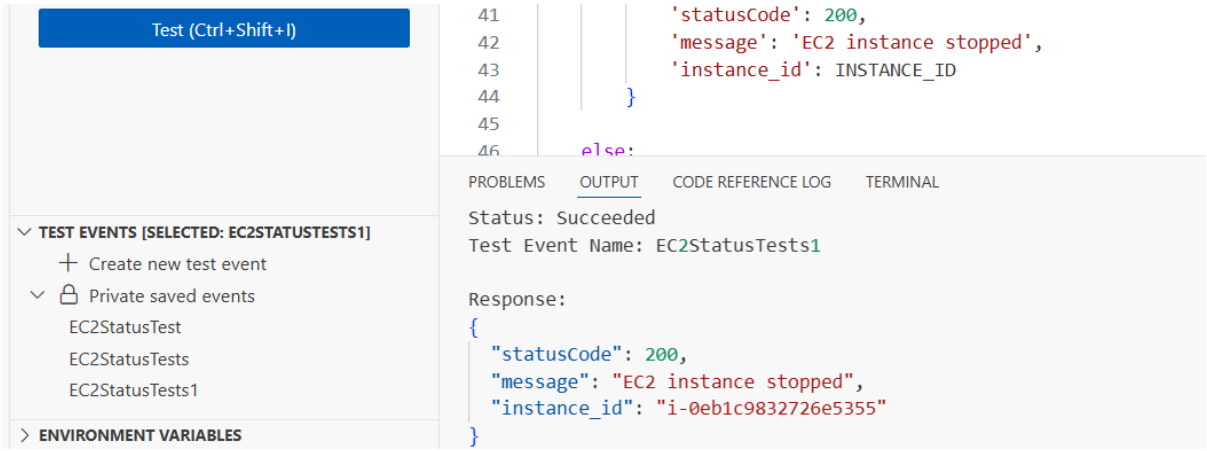
During testing, the execution log displayed:

```
Current EC2 state: stopped
EC2 instance i-0eb1c9832726e5355 is stopped. Starting it.
```

The Lambda response was:

```
{
    "statusCode": 200,
    "message": "EC2 instance started",
    "instance_id": "i-0eb1c9832726e5355"
}
```

This confirmed that the Lambda function successfully detected the stopped state and started the EC2 instance.



Step 7 – Create CloudWatch/EventBridge Schedule for 8:00 AM UTC

Amazon EventBridge was used to automatically trigger the Lambda function every day.

A schedule rule was created for the daily start operation.

Required time:

`8:00 AM UTC`

The cron expression used was:

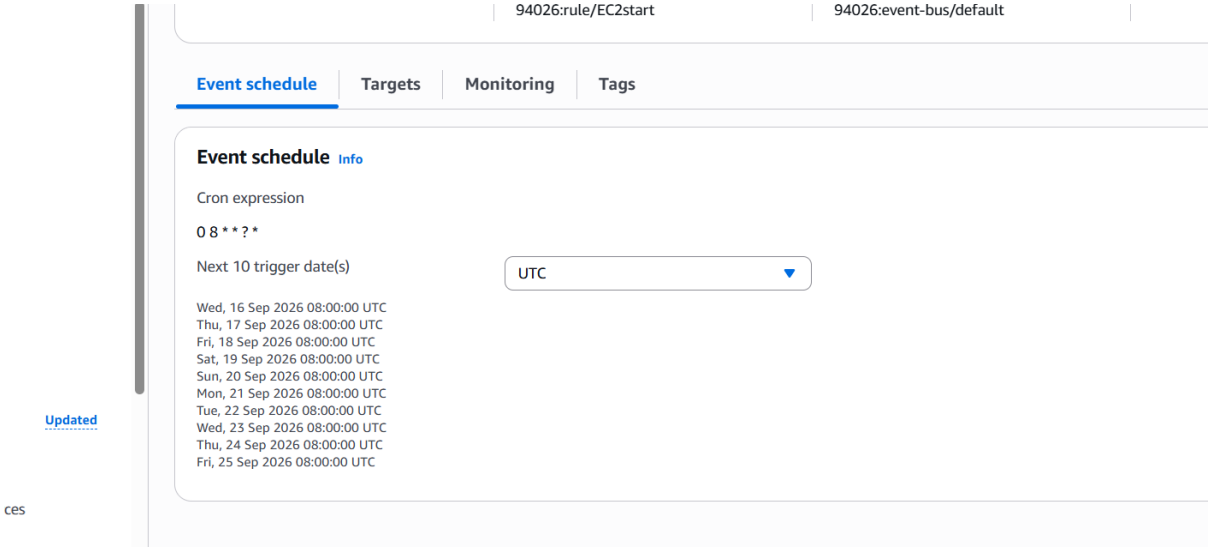
`cron(0 8 * * ? *)`

The target Lambda function was:

`EC2StatusChecker`

This rule triggers the Lambda every day at **8:00 AM UTC**.

If the EC2 instance is stopped, the Lambda starts the instance.



Step 8 – Create CloudWatch/EventBridge Schedule for 5:00 PM UTC

A second EventBridge schedule rule was created for the daily stop operation.

Required time:

`5:00 PM UTC`

The cron expression used was:

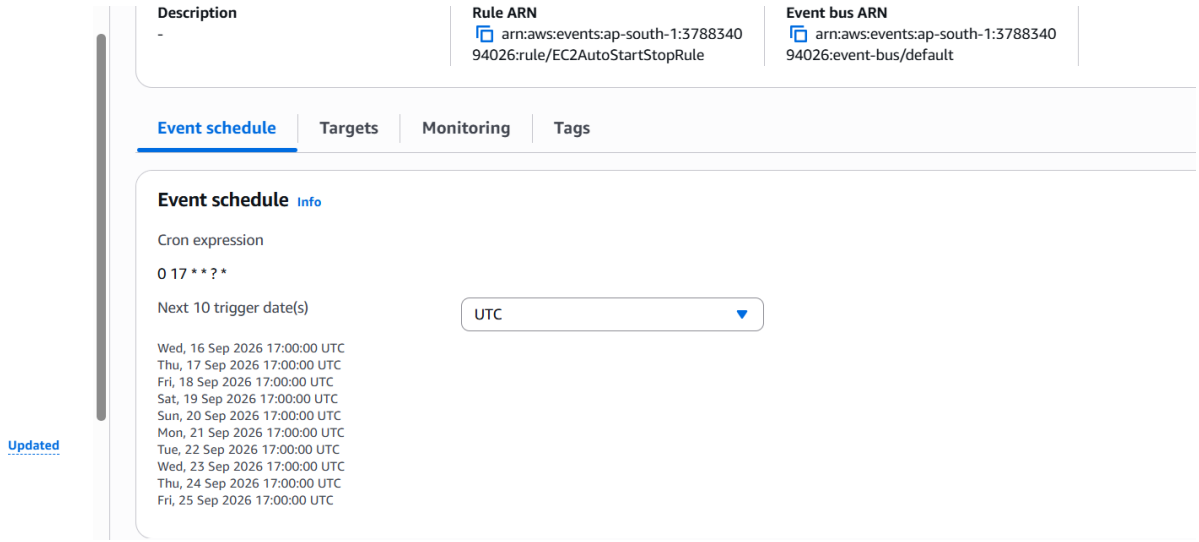
`cron(0 17 * * ? *)`

The target Lambda function was:

`EC2StatusChecker`

This rule triggers the Lambda every day at **5:00 PM UTC**.

If the EC2 instance is running, the Lambda stops the instance.



Step 9 – Final Automation Workflow

The final automation was configured as:

```
8:00 AM UTC
      ↓
EventBridge Rule
      ↓
Lambda
      ↓
Check EC2 State
      ↓
If STOPPED → START
```

and:

```
5:00 PM UTC
      ↓
EventBridge Rule
      ↓
Lambda
      ↓
Check EC2 State
      ↓
If RUNNING → STOP
```

Both EventBridge rules use the same Lambda function:

```
EC2StatusChecker
```

The same EC2 instance is controlled:

`i-0eb1c9832726e5355`

Verification

- The required IAM policy was successfully created.
- The policy included `ec2:DescribeInstances`, `ec2:StartInstances`, and `ec2:StopInstances`.
- The IAM role `ec2_role` was created and the policy was attached.
- The Lambda function `EC2StatusChecker` was successfully created using Python 3.14.
- The Lambda function successfully checked the EC2 instance state.
- The Lambda function successfully started the EC2 instance when its state was `stopped`.
- The Lambda function was tested successfully using an empty JSON event `{}`.
- An EventBridge rule was configured to trigger the Lambda every day at **8:00 AM UTC**.
- A second EventBridge rule was configured to trigger the Lambda every day at **5:00 PM UTC**.
- The 8:00 AM schedule uses:

Conclusion

Through this task, an automated EC2 start and stop process was successfully configured using **AWS IAM, Lambda, EC2, and Amazon EventBridge**. An IAM role was created with the required EC2 permissions, and a Python 3.14 Lambda function was developed using Boto3 to determine the current state of the EC2 instance and perform the appropriate start or stop operation. The Lambda function was tested successfully. Two daily EventBridge cron schedules were configured to trigger the Lambda at **8:00 AM UTC** and **5:00 PM UTC**. This task provided practical hands-on experience with IAM roles and policies, AWS Lambda, Boto3, EC2 automation, CloudWatch/EventBridge scheduling, and cron-based automation.